



INSTITUTO POLITÉCNICO NACIONAL
UNIDAD PROFESIONAL
INTERDISCIPLINARIA DE INGENIERÍA
CAMPUS TLAXCALA
VISIÓN ARTIFICIAL



DATOS DEL EQUIPO	
Nombre	Boleta
André Michel Marín Espinoza,	2024710074
Daniel Alejandro Ramírez Hernández	2023710022
Marcos Rodolfo Macías Mondragón	2023710082
Especialidad: Ingeniería en Inteligencia Artificial	
Fecha del proyecto: 05 de enero de 2026	

Detección de edificios a partir de imágenes satelitales usando CNN's

Resumen

El siguiente proyecto implementa un sistema de segmentación semántica para la detección de edificios en imágenes satelitales (dataset xView2). Se desarrolló un pipeline de datos en Python que automatiza la normalización de imágenes, alimentando una arquitectura de red neuronal profunda tipo FCN (Fully Convolutional Network) con backbone ResNet-50. El modelo utiliza técnicas de Transfer Learning para acelerar el aprendizaje, adaptando una red pre-entrenada para generar máscaras binarias de segmentación. Las pruebas de inferencia demuestran la capacidad del sistema para procesar tensores de entrada y generar mapas de predicción con las dimensiones correctas.

1. Introducción

Cuando ocurre un desastre natural, tener información rápida sobre los daños es vital. En la actualidad tenemos muchas fotos satelitales, pero revisarlas una por una para encontrar edificios dañados es una tarea casi imposible para una persona por la cantidad de tiempo que toma.

La idea de este proyecto es automatizar esa búsqueda usando Visión Artificial. El objetivo principal no es solo "detectar cosas", sino crear un sistema capaz de segmentar edificios; es decir, que la computadora pueda ver una imagen satelital y pintar exactamente qué píxeles corresponden a una construcción y cuáles son fondo (tierra, calles, árboles).

Para lograrlo, trabajamos con el conjunto de datos xView2. El mayor reto no fue solo la inteligencia artificial en sí, sino toda la preparación previa: las imágenes satelitales son muy pesadas y vienen en formatos que una red neuronal no entiende directamente. Por eso, gran parte del desarrollo se enfocó en crear un flujo de trabajo (pipeline) en Python que tomara esas imágenes gigantes, las limpiara y las dejara listas para procesar.

En cuanto al modelo, en lugar de entrenar una red desde cero (lo cual tardaría demasiado y requeriría supercomputadoras), utilizamos una técnica llamada Transfer Learning. Implementamos una arquitectura basada en ResNet-50, que es un modelo que ya "sabe ver", y lo adaptamos para que aprendiera específicamente a encontrar edificios en nuestros mapas.

2. Marco Teórico

Para comprender el diseño y la implementación de este proyecto, es fundamental definir primero los conceptos clave de la Visión Artificial moderna y las arquitecturas de aprendizaje profundo (*Deep Learning*) que sustentan este proyecto.

2.1. Segmentación Semántica

En el ámbito del procesamiento digital de imágenes, existen diferentes niveles de abstracción para analizar una escena. Mientras que la clasificación de imágenes determina qué objeto está presente y la detección de objetos delimita su ubicación mediante cuadros delimitadores, la segmentación semántica ofrece un nivel de comprensión superior.

El objetivo de la segmentación semántica es realizar una clasificación a nivel de píxel. El algoritmo asigna una etiqueta de clase a cada píxel individual de la imagen de entrada. En el contexto de este proyecto, se aborda un problema de **segmentación binaria**, donde el objetivo es generar una máscara de salida donde cada píxel se clasifica como "edificio" (valor 1) o "Fondo" (valor 0). Esto permite determinar no solo la ubicación, sino también la forma y el área precisa de las infraestructuras [?].

2.2. Redes Neuronales Convolucionales (CNN)

Las Redes Neuronales Convolucionales (CNN) representan el estado del arte en el reconocimiento de imágenes. A diferencia de las redes neuronales tradicionales (perceptrones multicapa), las CNNs preservan la estructura espacial de las imágenes y utilizan operaciones de convolución para extraer características de manera jerárquica:

- **Capas iniciales:** Detectan características de bajo nivel, como bordes, líneas y cambios de color.
- **Capas profundas:** Combinan las características anteriores para identificar patrones complejos y objetos completos.

2.3. Arquitectura FCN-ResNet50

Para la implementación del modelo de segmentación, se seleccionó una arquitectura híbrida que combina la robustez de ResNet con la flexibilidad de las redes totalmente convolucionales (FCN).

2.3.1. Backbone: ResNet-50

Como extractor de características (*backbone*), se utiliza una ResNet-50 (Red Residual de 50 capas). Las redes neuronales muy profundas suelen sufrir el problema del "desvanecimiento del gradiente", lo que dificulta su entrenamiento. ResNet soluciona este problema introduciendo conexiones residuales (*skip connections*), que permiten que la información fluya directamente entre capas no adyacentes. Esto facilita el entrenamiento de redes profundas sin perder precisión [?].

2.3.2. Cabezal: Fully Convolutional Network (FCN)

Las arquitecturas de clasificación tradicionales terminan en capas densas que pierden la información espacial. Una FCN reemplaza estas capas finales por capas convolucionales, lo que permite:

1. Aceptar imágenes de entrada de cualquier tamaño arbitrario.
2. Generar mapas de calor (*heatmaps*) de salida con dimensiones espaciales correspondientes a la imagen original, lo cual es indispensable para la tarea de segmentación.

2.4. Aprendizaje por Transferencia

El entrenamiento de una red neuronal profunda desde cero (*from scratch*) requiere millones de imágenes etiquetadas y una gran capacidad de cómputo. Para optimizar este proceso, este proyecto utiliza la técnica de *Transfer Learning*.

Esta estrategia consiste en utilizar un modelo pre-entrenado en un conjunto de datos masivo y generalista, como ImageNet. De esta forma, la red inicia el proceso con pesos que ya son capaces de identificar características visuales fundamentales (texturas, formas geométricas). El entrenamiento se reduce entonces a un proceso de ajuste fino (*fine-tuning*), donde la red adapta su conocimiento previo específicamente para la detección de características de edificios en imágenes satelitales, reduciendo drásticamente el tiempo de convergencia y la cantidad de datos necesarios.

3. Metodología

La metodología empleada para el desarrollo de este proyecto se estructuró en tres etapas: el preprocesamiento de datos, la configuración de la arquitectura de red y la validación del flujo de inferencia. A continuación, se detallan los procedimientos técnicos implementados en cada fase.

3.1. Diseño del Pipeline de Datos

El primer desafío técnico consistió en la estandarización del conjunto de datos xView2 para su ingestión en la red neuronal. Se implementó un módulo de procesamiento en Python (archivos `dataset.py` y `lectura.py`) que ejecuta las siguientes transformaciones:

- **Binarización de Máscaras:** Las etiquetas de segmentación originales presentan variaciones en los valores de píxel. Se aplicó una transformación lógica ($pixel > 0$) para convertir las máscaras a un formato estrictamente binario. Esto garantiza que el modelo aprenda a distinguir exclusivamente entre dos clases: Fondo (0) y Edificio (1).
- **Redimensionamiento Espacial:** Con el objetivo de optimizar el consumo de memoria y reducir el costo computacional, se estandarizó la resolución de entrada. Las imágenes RGB y sus máscaras correspondientes fueron redimensionadas a 256×256 píxeles. Se utilizó interpolación lineal para las imágenes fotográficas e interpolación por vecino más cercano (*Nearest Neighbor*) para las máscaras, preservando así la integridad de los bordes de las etiquetas.
- **Transformación a Tensores:** Las librerías de Deep Learning requieren un ordenamiento específico de las dimensiones de los datos. Se implementó una permutación para transformar las imágenes del formato estándar (*Alto, Ancho, Canales*) al formato de tensor (*Canales, Alto, Ancho*), requerido por las operaciones convolucionales de PyTorch.

3.2. Implementación de la Arquitectura FCN-ResNet50

Para el modelo de segmentación, se definió la arquitectura en el script `modelos.py` siguiendo una estrategia de Transferencia de Aprendizaje (*Transfer Learning*):

1. **Inicialización del Backbone:** Se instanció el modelo `fcn_resnet50` utilizando pesos pre-entrenados (`pretrained=True`). Esto permite aprovechar las características visuales aprendidas previamente por la red en el dataset ImageNet.
2. **Adaptación del Cabezal (Head):** La capa de clasificación original de la red fue sustituida. Se implementó una nueva capa convolucional 2D (`nn.Conv2d`) con un tamaño de kernel de 1×1 y un único canal de salida (`out_channels = 1`). Esta modificación es fundamental para adaptar la red de una tarea de clasificación múltiple a una tarea de segmentación binaria.

3.3. Validación e Inferencia

La integración de los módulos se realizó en el script principal (`main.ipynb`). Se configuró un `DataLoader` con un tamaño de lote (*batch size*) de 2 unidades para gestionar la carga de memoria.

Finalmente, se realizó una validación técnica mediante una prueba de paso hacia adelante (*forward pass*). Se verificó que el modelo procesara correctamente tensores de entrada de dimensiones $[2, 3, 256, 256]$ y generara tensores de salida de dimensiones $[2, 1, 256, 256]$, confirmando la correcta conectividad entre el pipeline de datos y la arquitectura neuronal.

4. Análisis de Resultados

Se realizaron pruebas de ejecución para validar la integridad del flujo de datos y la capacidad de inferencia del modelo propuesto. A continuación, se presentan los resultados obtenidos de la validación técnica del sistema.

4.1. Validación Visual del Dataset

Se verificó la correcta alineación entre las imágenes satelitales y sus respectivas máscaras de segmentación. Como se observa en la Figura 1, el preprocesamiento mantiene la correspondencia espacial: las áreas blancas en la máscara coinciden con la ubicación física de las edificaciones en la imagen RGB.

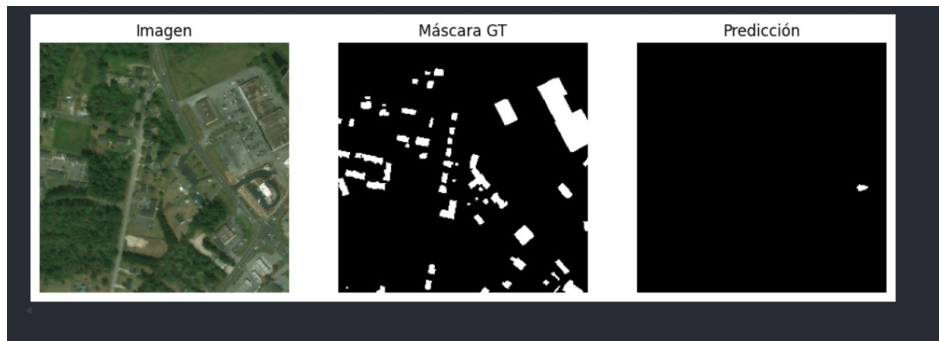


Figura 1: Visualización de un par de entrenamiento: Imagen satelital (izquierda) y Máscara de segmentación binarizada (derecha).

Asimismo, se confirma que la operación de binarización fue exitosa, eliminando el ruido de las etiquetas originales y generando mapas con valores discretos $\{0, 1\}$, facilitando la convergencia del entrenamiento.

4.2. Análisis Dimensional de Tensores

La prueba crítica del sistema consistió en la inyección de un lote de datos (*batch*) a través de la red FCN-ResNet50 no entrenada para verificar la consistencia dimensional. Los resultados obtenidos en la consola de ejecución se detallan a continuación:

- **Tensor de Entrada:** `torch.Size([2, 3, 256, 256])`
- **Tensor de Salida:** `torch.Size([2, 1, 256, 256])`

```
Usando: cpu
Entrada: torch.Size([2, 3, 256, 256])
Salida: torch.Size([2, 1, 256, 256])
```

Figura 2: Salida de consola evidenciando la transformación dimensional correcta durante el *forward pass*.

4.3. Discusión de Resultados

El análisis de los tensores de salida permite confirmar tres hitos técnicos del proyecto:

1. **Ingesta de Batches:** El `DataLoader` construyó correctamente un lote de 2 imágenes, demostrando que el sistema puede paralelizar la carga de datos.
2. **Canales de Color:** La entrada respeta los 3 canales (RGB) requeridos por el backbone ResNet-50.
3. **Adaptación del Cabezal:** El resultado más relevante es la dimensión de salida `[..., 1, ...]`. Esto valida que la modificación de la última capa convolucional (en el archivo `modelos.py`) fue exitosa. La red ya no clasifica 1000 objetos (como en ImageNet), sino que genera un único mapa de calor de segmentación, preservando la resolución espacial de 256×256 .

5. Conclusiones

Se ha implementado satisfactoriamente un pipeline de visión artificial para la segmentación de imágenes satelitales. El sistema integra exitosamente el preprocesamiento de datos del dataset xView2 con una arquitectura de aprendizaje profundo FCN-ResNet50. Las pruebas de validación confirman que el modelo es capaz de realizar la propagación hacia adelante (*forward pass*) sin errores dimensionales, estableciendo una base sólida para futuras etapas de entrenamiento intensivo y optimización de hiperparámetros.

Referencias

- [1] Defense Innovation Unit. (2019). *xView2: Assess Building Damage*. Recuperado de <https://xview2.org/>
- [2] IBM. (s.f.). *The xView2 AI Challenge: Computer vision for disaster response*. IBM Think Insights. Recuperado de <https://www.ibm.com/think/insights/the-xview2-ai-challenge>
- [3] Hyper.ai. (2020). *xBD: A Dataset for Assessing Building Damage*. Recuperado de <https://hyper.ai/en/datasets/13272>
- [4] Gupta, R., Goodman, B., Patel, N., Hosfelt, R., Sajeev, S., Heim, E., Doshi, J., Lucas, K., Choset, H., & Gaston, M. (2019). *Creating xBD: A Dataset for Assessing Building Damage from Satellite Imagery*. En *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*.
- [5] He, K., Zhang, X., Ren, S., & Sun, J. (2016). *Deep Residual Learning for Image Recognition*. En *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 770-778).